LER O CONTEÚDO

`cat` num log gigante arrasa a rolagem; `cat` num binário embaralha o terminal. Nenhum dos dois é acidente — o `cat` não serve para ver arquivo.

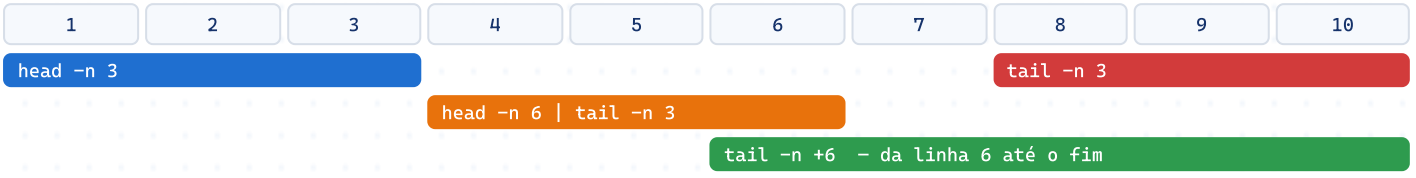
011
GUIAS LINUX

RODE `FILE` ANTES DE ABRIR — E ESCOLHA A FERRAMENTA

TEXTO CURTO	TEXTO GRANDE	SÓ AS PONTAS	CRESCENDO	COMPRIMIDO	BINÁRIO
`cat`	**`less`**	**`head`** **`tail`**	**`tail -F`** **`less +F`**	**`zcat zless zgrep`**	**`strings`** **`xxd`**
Cabe numa tela. Só aí.	Carrega sob demanda: 10 GB abrem na hora.	Primeiras ou últimas linhas.	Acompanha o arquivo em tempo real.	Lê o `.gz` sem descomprimir.	**Nunca** `cat` — conserto: `reset` .

O `cat` *concatena* — exibir é só o que sobra quando você dá um arquivo e não redireciona. O `file` não confia na extensão: lê os primeiros bytes e compara com um banco de assinaturas. Num binário, os bytes que por acaso formam sequências de escape VT100 são **obedecidos** — o terminal não travou, ele mudou de fonte e de mapa de caracteres; digite `reset` às cegas e Enter. E o `cat -A` é diagnóstico de verdade: revela `^M` (fim de linha do Windows — a causa do `bad interpreter: /bin/bash^M`), `^I` (tabulação onde você achava que havia espaços) e `$` (espaço sobrando no fim da linha).

O SINAL DE MAIS MUDA TUDO: AS FAIXAS DE `HEAD` E `TAIL`



DEZ TECLAS DO `LESS` E OS REFLEXOS QUE SOBRAM

TECLA	FAZ
`Espaço` / `b`	Avança / volta uma tela
`g` / `G`	Início / fim do arquivo
`/texto` · `n` `N`	Busca; próxima e anterior ocorrência
`&texto`	**Mostra só as linhas que casam** — um grep interativo sem sair do arquivo (`&` + Enter volta ao todo)
`F`	Acompanha o crescimento; `Ctrl+C` sai do modo **sem perder o histórico**
`m` +letra · `'` +letra	Marca a posição e volta a ela
`v` · `h` · `q`	Abre no `$EDITOR` · ajuda · sai

`tail -f` × `tail -F`
O `-f` segue o **inode**: depois que o `logrotate` renomeia o arquivo, ele fica olhando o antigo para sempre. O `-F` segue o **nome** e reabre. Em servidor é sempre `-F` .

`tail -F auth.log | grep --line-buffered "Failed"`
Sem `--line-buffered` o grep segura kilobytes e você fica minutos achando que travou.

`less -R` · `export LESS='-R -i -M'`
`-R` interpreta as cores do `nmap` ; sem ele você lê `ESC[1;32m` em cada linha. É o mesmo paginador do `man` .

`grep erro arquivo`
Não `cat arquivo | grep` : todo comando que aceita nome de arquivo dispensa o `cat` — é um processo e uma cópia de dados a menos.

`tac` · `batcat` · `jq .` | `less -R`
`tac` lê do fim para o começo. `batcat` (o `bat` do Debian) colore por sintaxe — ótimo para código, dispensável para dados que seguem num cano.

RESUMO RÁPIDO

- ✓ `file` antes de abrir: ele lê os bytes, não a extensão.
- ✓ `head -n 3` as três primeiras · `tail -n 3` as três últimas · `tail -n +3` da terceira em diante.
- ✓ Encadear `head | tail` recorta qualquer faixa do meio.
- ✓ `cat > nota.txt` grava até o **`Ctrl+D`** (fim de arquivo). `Ctrl+C` ali trunca o arquivo.
- ✓ `more` não volta e não busca para trás — use só onde não houver `less`.

NO KALI

A `rockyou.txt` tem **mais de 14 milhões de linhas**: `cat` nela é perder o terminal na certa — `zcat rockyou.txt.gz | head`, `wc -l`, `zgrep`. Log moderno é **binário**: `cat` em `/var/log/journal` não serve para nada, use `journalctl -u ssh`, `-f`, `-p err` (Vol 7). Deixe `tail -F /var/log/auth.log` numa segunda aba enquanto trabalha: **todo `sudo` seu aparece ali** — ensina rápido o que o sistema registra a seu respeito. Relatório de `nmap -oN` numa `/16` é enorme: abra no `less` e filtre com `&`. `strings` num binário suspeito é o **primeiro** passo — em VM isolada, sem executar, e só com material que você tem autorização para analisar.